NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

Centre for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI



L09
NLP &
Embeddings

From keyword search to semantic understanding

Copyright © 2026 Nanyang Technological University. All rights reserved.

Today's Three Questions

From characters to meaning

1

Why does keyword search break on real catalogues? (Notebook 01)

Synonyms, paraphrases, intent. The catalogue says 'frock'; the customer types 'dress'.

2

What IS an embedding? (Notebook 02, 03)

Computers don’t understand meaning the way humans do. Instead, words are represented as vectors, where **cosine similarity measures how closely their directions align**. Words with similar meanings tend to have vectors pointing in similar directions

3

How do we build production semantic search? (Notebook 04)

Pretrained **sentence-transformer** **all-MiniLM-L6-v2** (converts sentences into fixed-sized dense vectors) + **cosine similarity** (measures how similar two vectors are) + **structured filters**.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Why Keyword Search Fails (Notebook 01)

A concrete worked example

Customer query

"blue summer dress"

Catalogue entry (Lila Floral Sundress)

"Lightweight floral frock perfect for warm summer days. Tea-length, pastel print, breathable cotton."

Keyword overlap

Just 1 word: 'summer'. Misses dress (name says SUNDRESS; description says FROCK).
Misses blue (description says PASTEL).
No amount of stemming (reducing words to root form e.g. dresses to dress) or stop-words (e.g. filler words like “for”) fixes synonyms.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Early NLP: Words As Vectors — One-Hot Encoding

Key Idea: Used one-hot encoding to represent each word as a unique vector

① How it works

Vector size = vocabulary size

→ for e.g. 50K words = **50K+ dimensions**

One unique position per word

Each word gets one unique position in the vector

No overlap between words

Different words have no overlap → similarity = 0

Where it is used

Used in text classification and word matching

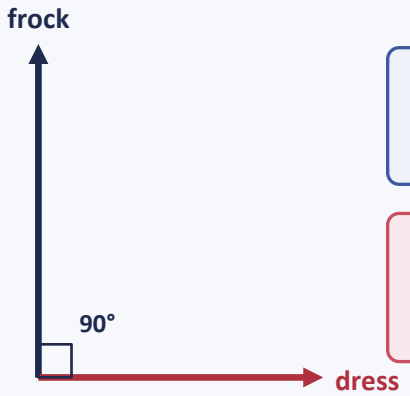
② One-hot in action

One-hot encoding

frock	→	0	0	**1**	0	… 0
dress	→	0	0	0	**1**	… 0
banana	→	0	0	0	0	… **1**

Cosine similarity

Measures the angle between two vectors



frock vs frock

angle 0° → similarity **1**

frock vs dress / banana

angle 90° → similarity **0**

Every different pair sits at 90°.

③ The problem

All words equally distant

All different words are equally far apart.

frock ≈ banana

in cosine similarity, just like frock ≈ dress

KEY LIMITATION

One-hot encoding captures *which word it is,*
not *what it means.*

Copyright © 2026 Nanyang Technological University. All rights reserved.

NLP Advancement: Capturing Meaning, Not Just Identity

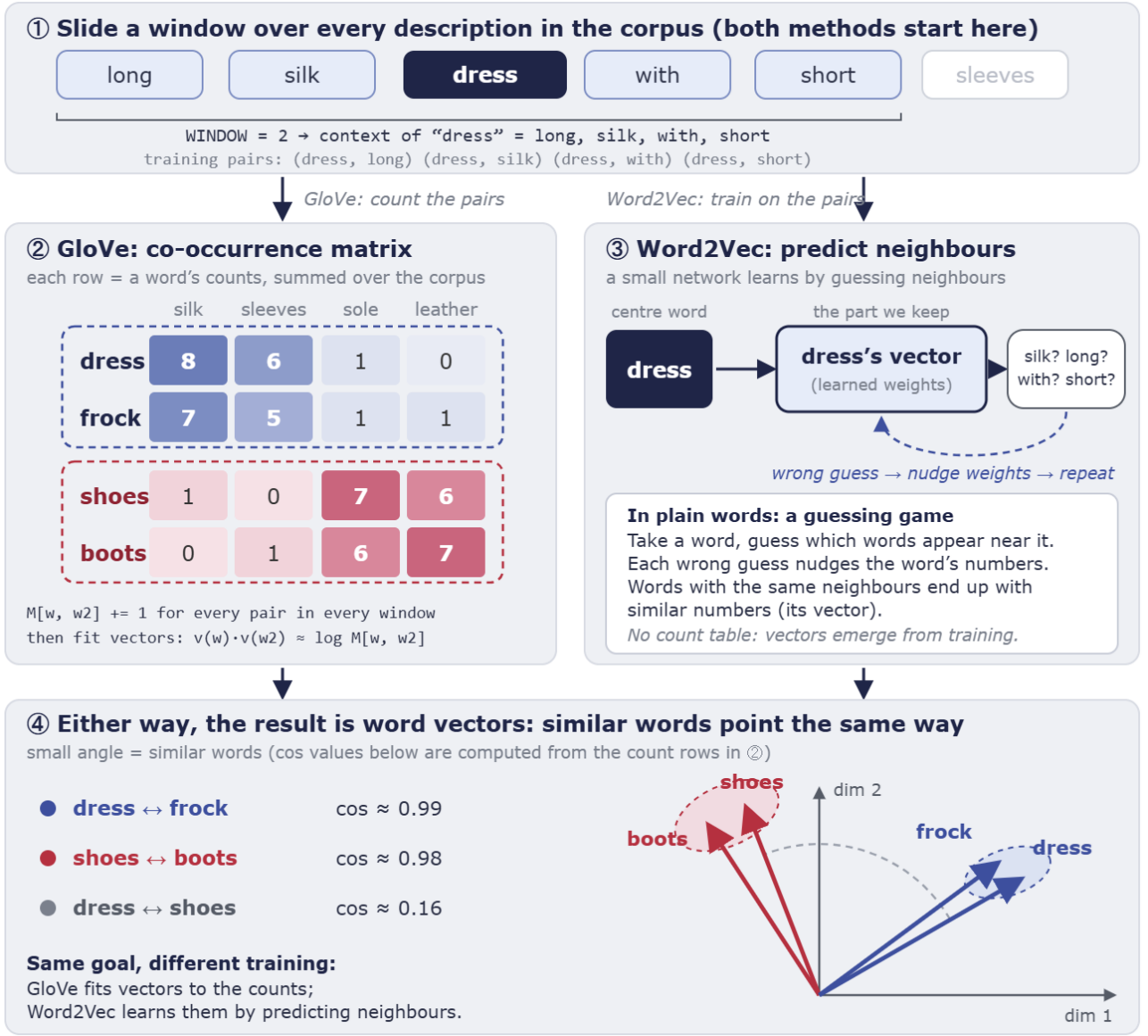
Key idea *(distributional hypothesis, Firth 1957): words used in* ***similar contexts*** *tend to mean* ***similar things***

GloVe: learns by counting

- Slides a fixed-size window across text, c**ounting how often words appear near one another**. It combines these counts into a global co-occurrence matrix, then learns vectors whose dot products reflect the log of the co-occurrence counts. *Example: "dress" and "frock" both frequently appear near "floral", "midi" and "elegant", so their vectors may be similar.*

Word2Vec: learns by predicting

- Slides a fixed-size window across text and trains a small neural network to **predict neighbouring words or a target word from its surrounding context.** The network's learned weights become word vectors. Words appearing in similar contexts develop similar vectors. Example: When the target word is "sneakers", Word2Vec learns to predict nearby words like "running" and "sports"; similarly, "trainers" predicts similar neighbouring words, so their vectors become similar.



Copyright © 2026 Nanyang Technological University. All rights reserved.

NLP Advancement: Capturing Meaning, Not Just Identity

① Real-life applications

Semantic Search & Recommendations

Find related words to improve search results and product recommendations.

e.g. dress → frock *related words*

Text Analysis

Support sentiment analysis and text classification by capturing relationships between words.

e.g. excellent → positive reviews

terrible → negative reviews

② Limitations

Static Word Embedding Limitation

Word2Vec and GloVe assign each word a fixed vector, so they **struggle to distinguish different meanings** of the same word depending on context.

e.g.



Synonym Identification

Word2Vec and GloVe rely on contextual similarity rather than explicit synonym relationships, so they **may fail to identify some synonyms or incorrectly treat related but distinct terms as interchangeable.**

e.g. dress ≈ skirt *related, but not interchangeable*

Copyright © 2026 Nanyang Technological University. All rights reserved.

Toy Embedding (Notebook 02)

Toy version of **GloVe**: count which words appear near each other.

Key idea: Represent each word as a **numerical vector** built from the company it keeps. Words that appear near similar words get **similar vectors**.

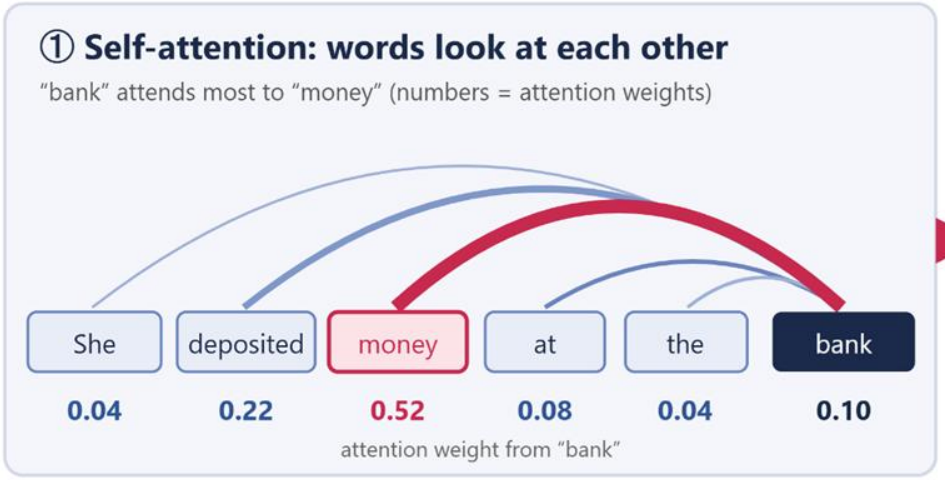
This notebook: Step 1: count co-occurrences in a sliding window (code below). Step 2: compress the counts into compact vectors (Extension E2).

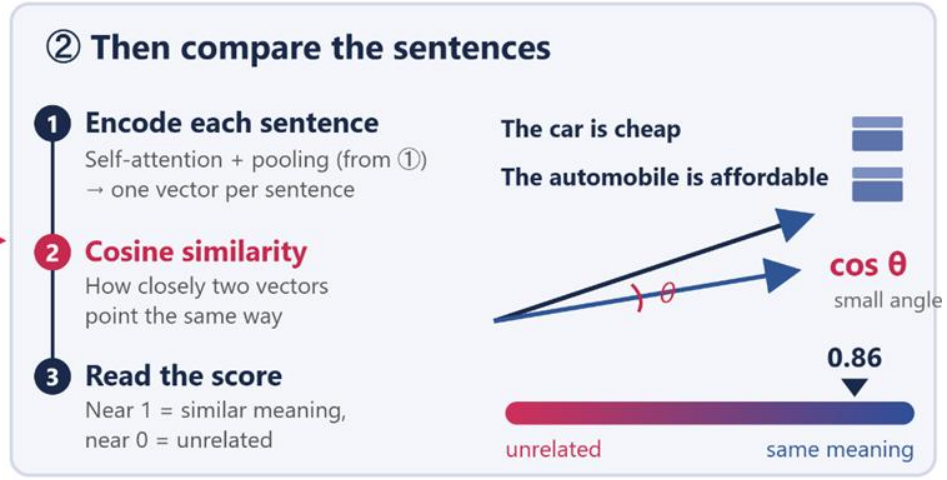
```
# ---- TOY GloVe, step 1: COUNT co-occurrences (step 2, compression, is Extension E2) ----
# Slide a small window across every description and count which words appear near each other.
# (Word2Vec uses this same window but PREDICTS the neighbours instead of counting them.)
def build_cooccurrence(documents, vocab, window=WINDOW):
    V = len(vocab)
    word_to_idx = {w: i for i, w in enumerate(vocab)}
    M = np.zeros((V, V), dtype=np.float32)
    for doc in documents:
        toks = tokens(doc)
        for i, w in enumerate(toks):
            if w not in word_to_idx: continue
            wi = word_to_idx[w]
            for j in range(max(0, i-window), min(len(toks), i+window+1)):  # the window around word i
                if j == i: continue
                w2 = toks[j]
                if w2 not in word_to_idx: continue
                M[wi, word_to_idx[w2]] += 1
    return M
```

Copyright © 2026 Nanyang Technological University. All rights reserved.

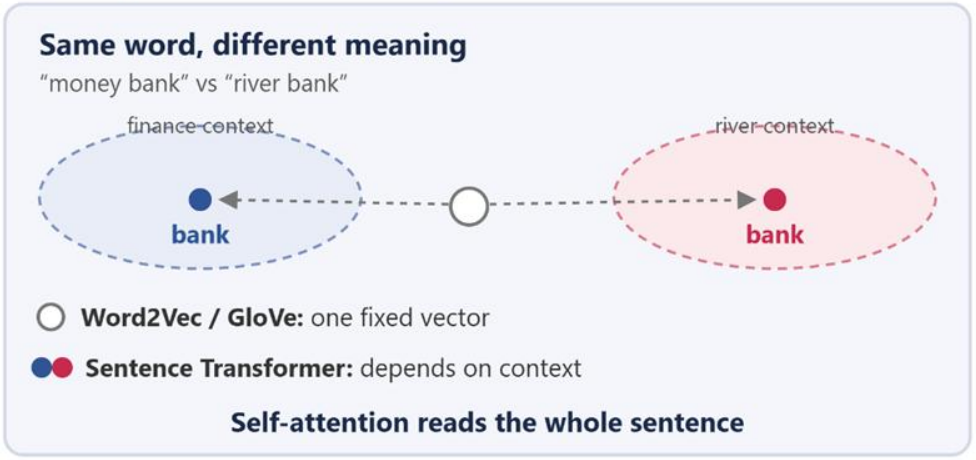
Modern NLP: Sentence Transformers with Contextual Embeddings

Key Idea: Turn text into dense vectors, whose meaning depends on relationships between words**. Self-attention weighs how much each word should attend to every other word, including itself.** These weights combine information from other words to create updated, context-aware representations. Across Transformer layers, these representations are refined and combined into sentence embeddings





③ Why it beats Word2Vec / GloVe





Copyright © 2026 Nanyang Technological University. All rights reserved.

Modern NLP: Sentence Transformers with Contextual Embeddings

① How it improves real-life applications

vis-à-vis Word2Vec/GloVe

Semantic Search & Recommendations

Unlike Word2Vec and GloVe's fixed word vectors, Sentence Transformers compare whole-text embeddings to match meaning across different wording.

e.g. affordable dress ≈ budget-friendly clothing

Text Analysis

Word2Vec and GloVe also learn from context, but give each word **one fixed vector**, so averaging them blurs negation. Sentence Transformers **read the whole sentence at once**, so context shapes every word, improving sentiment analysis and text classification.

e.g. **not excellent** ≠ excellent *different meanings*

② Limitations

Context and Domain Limitations

May misinterpret **specialised terminology, ambiguous text, or domain-specific meanings** if these are poorly represented in its training data.

poorly represented in training data → **misinterpreted**

Semantic Similarity Limitations

Can assign misleading similarity scores, because texts with shared topics, structure or learned patterns can still end up with similar embeddings despite different meanings. This can lead to irrelevant search results or recommendations.

e.g. The service was fast ≈ **The service was slow**

similar embeddings, different meanings

Copyright © 2026 Nanyang Technological University. All rights reserved.

Modern NLP: Sentence Transformers with Contextual Embeddings (Notebook 03)

MiniLM-L6-v2: A lightweight sentence-transformer model that converts text into 384-dimensional embeddings

Key Idea: Texts with similar meanings have vectors that are closer together in vector space, enabling efficient semantic search and retrieval

```
for a, b in pairs:
    emb_a, emb_b = model.encode([a, b])
    sim = float(np.dot(emb_a, emb_b))   # already
normalised
```

Pair	Toy (NB 02)	MiniLM-L6-v2 (NB 03)
frock ⇔ dress	0.035	**0.249**
frock ⇔ gown	0.158	**0.288**
sundress ⇔ dress	0.054	**0.509**

Copyright © 2026 Nanyang Technological University. All rights reserved.

Building A Semantic Search Engine

Key Idea: build a clean, reusable **search engine**

A SemanticSearch class stores the catalogue and its vectors once, so each search needs only the query

1

add() — embed once

Joins each product's name and description, then encodes all products in one batch, up front

```
docs = self.df[list(text_cols)]
        .agg(' — '.join, axis=1).tolist()
self.embeddings = self.model.encode(docs)    # done ONCE
```

2

Wire-up — reuse vectors

Fills in the catalogue and the cached 76 × 384 embeddings directly, so nothing is re-encoded

```
engine = SemanticSearch(model)
engine.df = df.reset_index(drop=True).copy()
engine.embeddings = catalogue_emb    # cached (76 × 384)
```

3

search() — rank by similarity

Embeds the query, scores it against every product with cosine similarity, returns the top-K

```
q = self.model.encode([query])
sims = cosine_similarity(q, self.embeddings)[0]
order = np.argsort(-sims)[:top]    # top-K
```

4

search_with_filters()

Narrows to products matching the filters (e.g. category='dress'), then ranks only that subset

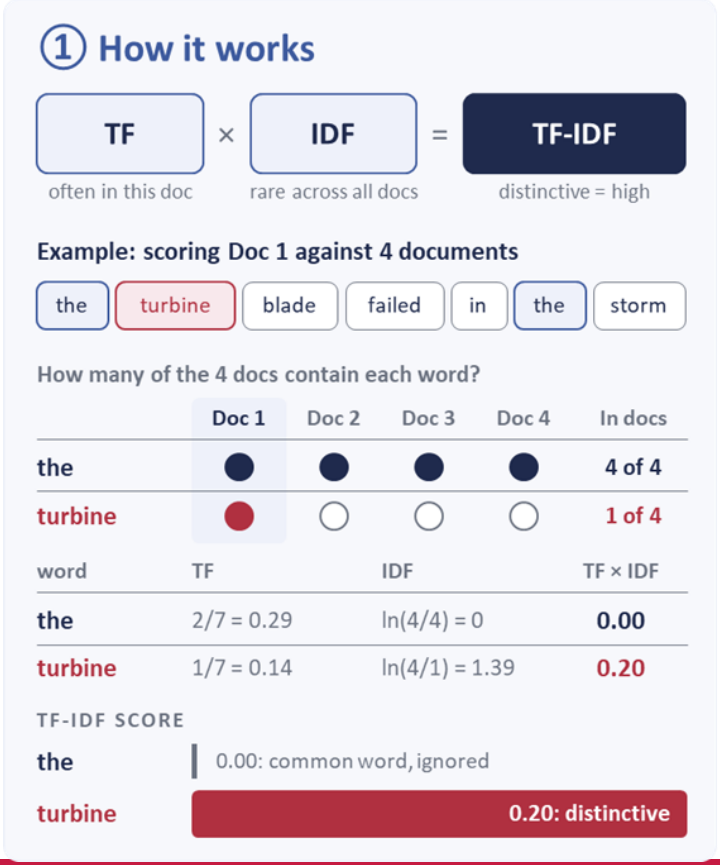
```
mask &= (self.df[col] == val)    # filter first
sub_emb = self.embeddings[mask.values]
sims = cosine_similarity(q, sub_emb)[0]    # rank the subset
```

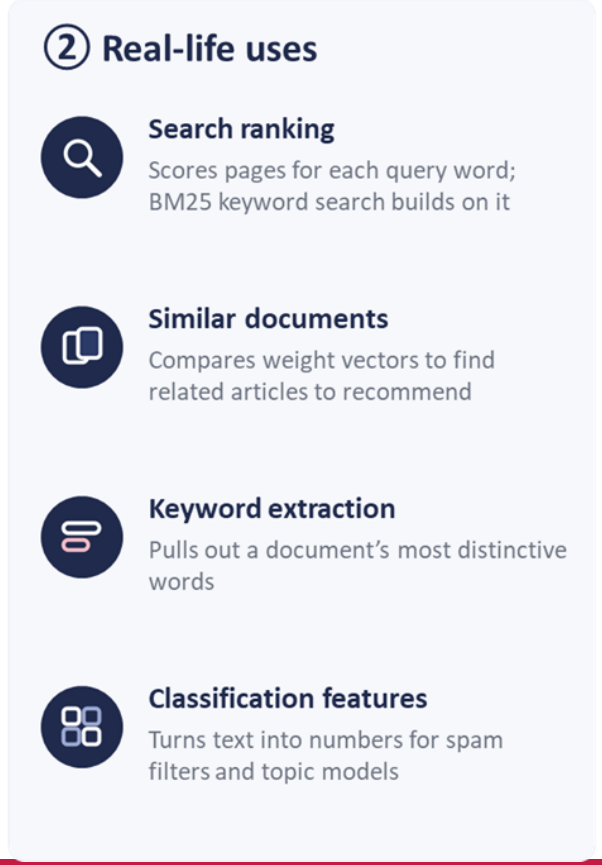
Copyright © 2026 Nanyang Technological University. All rights reserved.

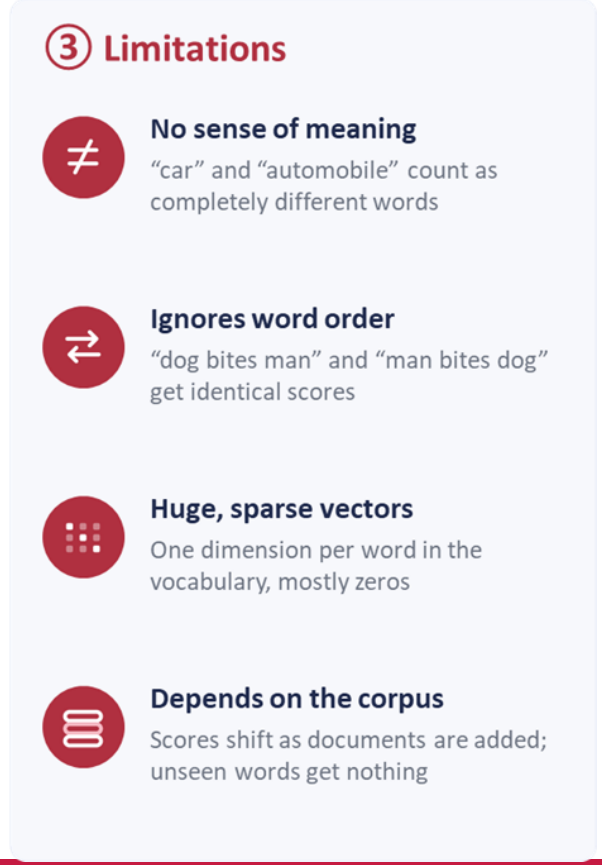
Benchmark Comparison: TD-IDF

TF-IDF (Term Frequency–Inverse Document Frequency): A numerical technique used in natural language processing (NLP) to measure the importance of a word in a document relative to a collection of documents

Key Idea: Words that appear frequently in a particular document but rarely across the entire collection receive higher scores, while common words across many documents receive lower scores







Copyright © 2026 Nanyang Technological University. All rights reserved.

Semantic vs TF-IDF (Notebook 04)

Head-to-head on the 8-query benchmark

Approach	Top-1 accuracy	Top-5 accuracy
Keyword (NB 01, stemmed)	5/8 (62%)	–
TF-IDF	4/8 (50%)	4/8 (50%)
Semantic (MiniLM)	6/8 (75%)	7/8 (88%)

Where TF-IDF and semantic each win

- TF-IDF wins on EXACT terms (SKUs, brand names, exact product names)
- Semantic wins on SYNONYMS, paraphrases, intent (frock/dress, gym/yoga, cosy/warm)
- Production: combine BOTH — hybrid retrieval

Copyright © 2026 Nanyang Technological University. All rights reserved.

Three Things To Remember

L09 takeaways

1

Embeddings turn discrete tokens into dense vectors where distance = meaning

Same garment, different words → similar vectors. Synonyms cluster.

2

Pretrained sentence-transformers are the default starting point

Don't train from scratch. Load all-MiniLM-L6-v2 and call .encode(). Two lines.

3

Production search is HYBRID, not pure semantic

Combine TF-IDF + semantic + structured filters + re-rank. Each component has its niche.

Copyright © 2026 Nanyang Technological University. All rights reserved.

Bridge to L10

Opening the transformer black box

We've used embeddings as a black box. But where do they come from?

"How does a model KNOW that frock and dress are similar without ever being told? The answer is attention — invented in 2017, behind GPT, BERT, ChatGPT, all of it."

What L10 covers

▸ **Attention**	The mechanism that lets a model decide which words to focus on for each output token.
▸ **Transformer block**	Multi-head attention + feed-forward + residual + layer-norm. The Lego brick of modern NLP.
▸ **Scaling**	Same architecture from MiniLM (22M params) to GPT-4 (~1T). The same recipe, bigger pots.
▸ **GenAI today**	ChatGPT, Claude, Gemini, Llama — all transformers. By Friday you'll understand the basics.

Copyright © 2026 Nanyang Technological University. All rights reserved.